26. Hashing and Bloom Filters (Infra-Relevant)

26. Hashing and Bloom Filters (Infra-Relevant)

Phase: Phase 4 — Algorithms and data structures
Estimated time: 1–2 days
Slug: `26-hashing-bloom`

Learning objectives

- Explain hash maps vs cryptographic hashes
- Implement a simple bloom filter
- Reason about false positives
- Apply hashing to caches and shards

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Hashing underpins maps, caches, and probabilistic filters. Bloom filters trade a false-positive rate for tiny memory — classic infra tool.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Hash functions distribute keys; collisions are normal.
- Go maps hide hashing; understand load and resizing conceptually.
- Bloom filters: insert sets bits; query may false-positive, never false-negative (for basic blooms).
- Choose k hash functions / bitset size from expected n and error rate.
- Cryptographic vs non-crypto hashes — don't confuse them.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-hashing && cd /tmp/gw-hashing
go mod init example.com/scratch
```

Then create `main.go`:

```go
package main

import (
    "fmt"
    "hash/fnv"
)

func hash(s string) uint64 {
    h := fnv.New64a()
    _, _ = h.Write([]byte(s))
    return h.Sum64()
}

func main() { fmt.Println(hash("go") != 0) }
```

Run it:

```
go run .
```

Expected output:

```
true
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Hash strings with FNV and print distribution buckets.
2. Sketch a tiny bitset Bloom with two hashes.
3. Demonstrate a false positive on purpose with a tiny filter.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Hash tables (maps)

Go's `map` is a hash table: average O(1) lookup. Hash collisions are handled internally.

Cryptographic vs non-crypto

- `hash/fnv`, `maphash`: fast, non-crypto
- `crypto/sha256`: integrity / security

Do not use non-crypto hashes for security tokens.

Bloom filters

Probabilistic set: can say "possibly in set" or "definitely not". Uses bit array + k hash functions. False positives; no false negatives (for standard bloom).

Used in: databases (skip disk reads), caches, distributed systems membership tests.

Tradeoff: memory vs false positive rate.

Failure modes to watch while coding

- Using MD5 for security (broken)
- Ignoring bloom false positives in logic
- Rolling your own crypto

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- hash() randomization in Python vs stable FNV here.
- set membership is exact; Bloom is probabilistic.
- Don't use MD5 for security; fine for demos of distribution.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

In caches, hashing also means key design: canonical serialization before hash.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Hashing and Bloom Filters (Infra-Relevant)" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Using MD5 for security (broken)
- Ignoring bloom false positives in logic
- Rolling your own crypto

Exercises

Work in `exercises/26-hashing-bloom/`. Read that folder's `README.md` before you code.

What you will build

Implement the bloom/hash stub per tests (likely Add/MightContain).

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/26-hashing-bloom/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What can a bloom filter never say?
2. When use sha256 vs fnv?
3. Why maps are O(1) average?

Further reading

- https://pkg.go.dev/hash
- https://en.wikipedia.org/wiki/Bloom_filter